

Project Report

INTRODUCTION

This report presents an **intelligent cleaning agent** operating in a linear environment of N rooms. The agent starts with **$2.5 \times N$ energy units** and faces three challenges:

- Limited energy (suck=dirt level 1-5, move=2)
- Dynamic re-dirtying (10% chance per timestep)
- Cleaning vs movement decisions.

Report structure:

1. Explanation of design decisions - Architecture overview
2. Agent behavior summary - Perception-action cycle
3. Performance observation - Energy usage and limitations analysis

Explanation of Design Decisions

Environment Class

The Environment class models the linear world with the following attributes:

Room States: Integer array `self.rooms` where 0=clean, 1-5=dirty level (compact encoding)

Dirtiness levels: An integer that encodes both the state and cleaning cost

Dynamic Behavior: `update_environment()` implement 10% re-dirtying mechanism per timestep

validation: $0 \leq \text{room_index} < \text{self.num_rooms}$ boundary checks

Key methods:

- `clean_room()` - room cleaning
- `update_environment()` - environmental dynamics simulation
- `get_room_state()` - percept seeing if it is dirty and the level of it (`is_dirty`, `dirtiness_level`)
- `is_all_clean()` - check the condition for the termination
- `randomize_initial_state()` - 50% rooms start dirty (realistic challenge)
- `set_room_dirtiness()` - safe room updates with boundary checks

Agent Class

The Agent class handles all agent behavior with three main parts:

1. Perception

Perceive() method gives agent exactly what assignment requires

- Current room number
- Is room dirty?
- Dirty level (if dirty)
- Energy left

2. Actions

- suck() - cleans current room (cost dirt level: 1-5 units)
- move_left() - moves left (costs 2 units)
- move_right() - moves right (costs 2 units)
- can_suck(), can_move() - check energy before acting (no invalid moves)
- No action if no energy

Decision Rules:

The baseline strategy implements a prioritized approach where cleaning the current room takes precedence over movement, with rightward movement preferred over leftward movement. The agent terminates when no meaningful actions remain possible.

1. Clean if current room dirty AND enough energy
2. Move right (unless at last room, then move left)
3. Stop if can't do anything

3. Simulation (run_simulation function):

Controls the entire simulation step-by-step:

- Reads: N (num_rooms) rooms, T(max_timesteps) steps from command line
- Loop: see → act → update rooms
- Stops: all clean OR no energy OR T steps
- Shows: print_results() with all required outputs

Agent Behavior Summary

The agent implemented in this simulation is a reactive, rule-based agent operating in a linear environment of N rooms. Its behavior is governed by the following rules:

1. Perception: At each timestep, the agent observes:

- Its current room index
- Whether the current room is dirty or clean

- The dirtiness level of the room (if dirty)
- Its remaining energy

2. Decision Making:

- **If the current room is dirty** and the agent has enough energy to perform the cleaning action (**Suck**), it cleans the room.
- **Otherwise**, the agent attempts to move. Movement is directional:
 - If at the last room, the agent moves left.
 - If at any other room, the agent moves right.
- Each movement consumes a fixed amount of energy (2 units).

3. Energy Management:

- Cleaning consumes energy equal to the dirtiness level of the room (1–5 units).
- Movement consumes 2 energy units per step.
- The agent can only perform actions if it has sufficient energy.

4. Termination:

- The agent stops when one of the following conditions is met:
 - All rooms are clean and the agent has no meaningful action left.
 - The agent runs out of energy and cannot perform any action.
 - The maximum number of timesteps **T** is reached.

5. Limitations:

- The agent has **no memory of previously visited rooms**. It does not plan its movement or prioritize rooms based on dirtiness or energy efficiency.
- Movement is purely linear, which can result in repeated traversal of the environment and unnecessary energy consumption.
- The agent does not account for the stochastic re-dirtiness of rooms beyond reacting to the current percept.

Performance Observations

Energy Usage

A significant portion of the agent's energy is consumed by movement rather than cleaning. Each move costs 2 units, and the agent frequently traverses the environment even through rooms that are already clean. Since movement is linear and unconditional when the agent cannot clean, energy is often wasted on revisiting rooms. This effect increases as the number of rooms grows, reducing energy available for cleaning.

Cleaning Effectiveness

The agent reliably cleans a room whenever it encounters dirt and has sufficient energy to execute the *Suck* action. However, because the agent follows a fixed movement policy and does not strategically revisit rooms, some rooms may remain dirty when the simulation terminates due to energy exhaustion or reaching the maximum timestep limit.

Additionally, the reported `rooms_cleaned_count` includes **multiple cleanings of the same room** caused by stochastic re-dirtiness. While this reflects correct interaction with the environment, it also indicates that the agent does not explicitly aim to maximize the number of *distinct* rooms cleaned, which limits overall cleaning efficiency.

Impact of the Dynamic Environment

The environment's stochastic behavior where clean rooms have a 10% probability of becoming dirty again has a significant impact on observed performance. Rooms cleaned early in the simulation may become dirty after the agent has moved away. Because the agent does not monitor or prioritize revisiting previously cleaned rooms, the environment may fail to reach a fully clean state even when initial cleaning coverage is high.

This highlights a key limitation of a **purely reactive agent operating in a dynamic environment**, where responsiveness alone is insufficient to maintain long-term goal satisfaction.

Scalability

As the number of rooms increases, observed performance degrades noticeably. Although the initial energy budget scales linearly with the number of rooms ($2.5 \times N$), the cumulative cost of movement across a larger environment often dominates energy consumption. Consequently, for larger values of N and T , the agent frequently exhausts its usable energy before achieving a stable clean configuration.

Overall Assessment

Overall, the baseline agent demonstrates **correct and robust interaction with the environment** and adheres to all specified rules. From a performance perspective, however, the absence of planning or heuristic reasoning results in suboptimal energy usage and limited adaptability to environmental dynamics. These observations suggest that incorporating planning strategies such as prioritizing highly dirty rooms, reducing unnecessary movement, or selectively revisiting rooms could substantially improve rational performance under the same constraints.

CONCLUSION

This baseline reflex agent correctly implements all specification requirements but reveals classic limitations of reactive behavior: excessive movement costs and poor handling of dynamic re-dirtying. Future work could add memory (model-based agent) or heuristics (prioritize dirty rooms) for better performance.